Zbirka zadataka: Databricks korak po korak

Praktične vežbe na Databricks Free Edition platformi, uz tačne korake kroz korisnički interfejs, u priči o firmi BrzaKorpa.

Kome je namenjena

Zbirka je namenjena polaznicima kursa Data Engineer. Svaki lab prati jedan čas iz plana i vodi korak po korak kroz ono što se na času radi u Databricks-u: gde se klikne, šta se upiše i kako izgleda rezultat. Lab-ovi se rade na času, u paru ili samostalno, a mogu da se ponove i kod kuće.

Kako se koristi

Svaki lab ima isti raspored: uz koji čas ide i koliko traje, kratku postavku iz priče, cilj, korake, očekivani rezultat, rešenja za najčešće zastoje i dodatni izazov za one koji završe ranije. Koraci se rade redom. Nazivi iz korisničkog interfejsa (dugmad, meniji, kartice) napisani su masnim slovima, a putanja kroz meni zapisuje se sa znakom ">" (npr. Settings > Preferences). Kod se kopira iz sivih okvira.

Lab-ovi se nadovezuju: tabele iz Lab-a 2 koriste se u svim kasnijim lab-ovima, a job, dashboard i alert rade nad slojevima toka koji se na Času 28 prave u notebook-ovima 01_bronze, 02_silver i 03_gold. Kada nešto ne radi, prvo se pogleda deo "Ako zapne", a zatim se pomoć traži Zoom reakcijom, pa pomaže predavač ili asistent.

Koraci odgovaraju stanju platforme i zvanične dokumentacije iz septembra 2026. Databricks povremeno menja nazive u meniju; tabela starih i novih naziva je u prilogu.

Pre početka

Za lab-ove su potrebni nalog na Databricks Free Edition (prvi lab), fajlovi sa podacima o dostavama i nalog na GitHub-u sa repo-om iz prvog časa. Fajlovi (dostave.csv, zone_dostave.csv, kurirske_sluzbe.csv, folder sa dnevnim fajlovima i ostali) dobijaju se od predavača kao zip arhiva, koja se pre prvog lab-a raspakuje na računaru, jer Databricks ne učitava zip arhive.

Pregled lab-ova

Lab	Tema	Uz čas	Trajanje
1	Nalog i prvi obilazak platforme	1	20 min
2	Tabele iz fajlova, bez koda	3	20 min
3	Prvi Python notebook	13	15 min

Lab	Tema	Uz čas	Trajanje
4	Fajlovi pored notebook-a	16	15 min
5	Unity Catalog u Catalog Explorer-u	24	15 min
6	Genie Code u notebook-u	24	20 min
7	Notebook-ovi u GitHub repo-u (Git folder)	24	30 min
8	Volume raw i dnevni fajlovi	25	30 min
9	Dnevni fajlovi kao tabela, u SQL-u i u Python-u	25	30 min
10	Alert za loš fajl kurirske službe	28	15 min
11	Dashboard za menadžment	29	25 min
12	Genie Agent, pitanja o dostavama na običnom jeziku	29	20 min
13	Job koji pokreće tok sam	29	35 min
14	Lakeflow pipeline sa proverama kvaliteta	34	35 min
15	Povezivanje dbt-a sa Databricks-om	36	20 min
16	AI Playground sa alatom	45	25 min

Lab 1: Nalog i prvi obilazak platforme

Uz čas: Čas 1 **Trajanje:** 20 min

> **Postavka:** Prvi dan na novoj platformi. Pre nego što bilo šta napravi, polaznik treba da zna gde se šta nalazi, kao kad prvi put uđe u magacin BrzaKorpe.

Cilj: polaznik otvara Free Edition nalog i zna čemu služi svaki deo menija sa leve strane.

Koraci

1. Otvori stranicu za registraciju login.databricks.com/?intent=CE_SIGN_UP i izaberi prijavu za **Free Edition** (preko Google ili Microsoft naloga, ili preko mejla sa jednokratnim kodom). Databricks sam napravi workspace.

2. Na početnoj strani pogledaj polje za pretragu i pitanja na vrhu: tu se pretražuju objekti u workspace-u i postavljaju pitanja na običnom jeziku.

3. Pogledaj meni levo. U grupi na vrhu su, između ostalih, **Workspace** (tvoji fajlovi i notebook-ovi), **Catalog** (podaci), **Jobs & Pipelines** (automatizacija) i **Compute** (računarski resursi).

4. Spusti se na grupu **SQL**: **SQL Editor**, **Queries**, **Dashboards**, **Genie Agents**, **Alerts**, **SQL Warehouses**.

5. Pogledaj grupu **AI/ML**, u kojoj su, između ostalih, **Playground**, **Agents** i **Serving**. Ovo koristimo u poslednjoj nedelji kursa.

6. Klikni **+ New** gore levo i samo pogledaj šta sve može da se napravi (notebook, query, dashboard, job, pipeline). Ne pravi ništa, zatvori meni.

7. Klikni na svoje ime gore desno > **Settings** > **Preferences** i, ako želiš, uključi tamnu temu.

Očekivani rezultat

Polaznik ume da pokaže gde su Catalog, SQL Editor, Jobs & Pipelines i dugme **+ New**.

Ako zapne

- Nema koda u mejlu: proveri folder za neželjenu poštu (spam) ili izaberi prijavu preko Google naloga.
- Stranica je na drugom jeziku: jezik se menja u Settings > Preferences.

Dodatni izazov

Nađi u meniju mesto gde se vide pokrenuti upiti (istorija upita). Kako se zove?

Rešenje: SQL > Query History.

Lab 2: Tabele iz fajlova, bez koda

Uz čas: Čas 3 **Trajanje:** 20 min

> **Postavka:** Šef data tima šalje tri fajla: dostave za januar, šifarnik zona i šifarnik kurirskih službi. Da bi ceo tim mogao da ih koristi, fajlovi treba da postanu tabele.

Cilj: polaznik pravi tri tabele iz fajlova samo preko korisničkog interfejsa, bez ijedne linije koda.

Koraci

1. Proveri da je zip arhiva sa podacima raspakovana: fajlovi dostave.csv, zone_dostave.csv i kurirske_sluzbe.csv treba da budu u običnom folderu na računaru.
2. Klikni **+ New** > **Add or upload data** > **Create or modify a table**.
3. Prevuci fajl **dostave.csv** u polje za upload (ili klikni **browse** i izaberi ga).
4. Gore desno je izabran SQL warehouse za pregled podataka (**Serverless Starter Warehouse**); ostavi ga.
5. Proveri odredište: katalog **workspace**, šema **default**, ime tabele **dostave**.
6. Klikni **Advanced attributes** i proveri da su uključeni **First row contains the header** i **Automatically detect column types** (podrazumevano jesu), pa zatvori prozor.
7. Pogledaj pregled podataka: kolone pickup_datetime i dropoff_datetime treba da budu prepoznate kao datum i vreme (ikonica sata uz naziv kolone), a delivery_fee kao broj.
8. Klikni **Create table** na dnu strane.
9. Ponovi korake 2-8 za **zone_dostave.csv** (tabela **zone_dostave**) i za **kurirske_sluzbe.csv** (tabela **kurirske_sluzbe**).
10. U **SQL Editor**-u pokreni proveru:

```
SELECT COUNT(*) FROM workspace.default.dostave;
```

Očekivani rezultat

U Catalog > workspace > default postoje tabele dostave, zone_dostave i kurirske_sluzbe, a upit iz koraka 10 vraća 100.400.

Ako zapne

- Kolone se zovu _c0, _c1: pod **Advanced attributes** nije uključeno First row contains the header.
- Svi tipovi su string: pod **Advanced attributes** uključi Automatically detect column types.
- Fajl ne prolazi: upload preko korisničkog interfejsa prima CSV, TSV, JSON, Avro, Parquet i TXT, bez zip arhiva.

Dodatni izazov

Otvori tabelu zone_dostave i odgovori: koliko ima redova i koje su kolone?

Rešenje: kolone se vide na strani tabele u Catalog Explorer-u, Catalog > workspace > default > zone_dostave (location_id, region, zone), a broj redova daje upit SELECT COUNT(*) FROM workspace.default.zone_dostave (51).

Lab 3: Prvi Python notebook

Uz čas: Čas 13 **Trajanje:** 15 min

> **Postavka:** Deo podataka BrzaKorpe ne živi u bazi, pa šef tima traži da se pravila, kao što je tarifa dostave, zapišu u Python-u. Prvi korak je notebook u kome se kod piše i odmah proverava.

Cilj: polaznik pravi Python notebook, povezuje ga sa serverless compute-om, piše i pokreće ćelije sa kodom i tekstom.

Koraci

1. U meniju levo klikni **Workspace**, pa otvori svoj lični folder (u delu Users, folder sa tvojom mejl adresom).
2. Klikni **+ New** > **Notebook**. Otvara se nov, prazan notebook.
3. Klikni na naziv notebook-a na vrhu strane i upiši novo ime: prvi_python.
4. Proveri da je jezik notebook-a Python (padajući meni u traci sa menijima, odmah posle menija Help).
5. Proveri da je notebook povezan sa serverless compute-om (meni za compute gore desno). Ako nije, izaberi **Serverless**.
6. U prvu ćeliju upiši print("Zdravo, BrzaKorpa") i pokreni je kombinacijom tastera Shift i Enter. Rezultat se vidi ispod ćelije.
7. Ispod ćelije dodaj novu ćeliju sa kodom, upiši cena = 179 + 22 * 6.69 i pokreni je. Zatim u sledećoj ćeliji upiši print(cena) i pokreni i nju: promenljiva iz prethodne ćelije je i dalje tu.
8. Dodaj ćeliju sa tekstom i upiši kratak opis notebook-a (npr. "Tarifa dostave, prvi pokušaji"). Tekstualne ćelije služe za objašnjenja uz kod.

Očekivani rezultat

U ličnom folderu postoji notebook prvi_python sa bar tri ćelije sa kodom i jednom ćelijom sa tekstom, a sve ćelije sa kodom su pokrenute bez greške.

Ako zapne

- Ćelija se ne pokreće i čeka: compute se još pokreće; posle nekoliko sekundi pokušaj ponovo.
- NameError: name 'cena' is not defined: ćelija u kojoj je promenljiva napravljena nije pokrenuta. Pokreni ćelije redom, odozgo nadole.
- Notebook je na jeziku SQL: jezik se menja u padajućem meniju za jezik, u traci sa menijima. Nov notebook dobija jezik koji je poslednji korišćen.

Dodatni izazov

Pokreni sve ćelije notebook-a odjednom, od prve do poslednje. Gde se nalazi ta komanda?

Rešenje: dugme **Run all** na traci notebook-a pokreće sve ćelije, redom.

Lab 4: Fajlovi pored notebook-a

Uz čas: Čas 16 **Trajanje:** 15 min

> **Postavka:** Kurirske službe šalju šifarnike i spiskove dostava kao fajlove. Da bi ih Python kod u notebook-u pročitao, fajlovi treba da budu u istom folderu kao i notebook.

Cilj: polaznik otprema fajlove u folder u kome je notebook i čita ih iz Python-a relativnom putanjom.

Koraci

1. U meniju levo klikni **Workspace** i otvori lični folder u kome je notebook prvi_python.
2. U tom folderu klikni **Create** > **Folder** i napravi folder fajlovi_python. Folder u Workspace-u je isto što i folder na računaru: u njemu će biti i notebook i fajlovi koje notebook čita.
3. Otvori folder fajlovi_python i tek onda u njemu napravi nov Python notebook (**Create** > **Notebook**) sa imenom citanje_fajlova.
4. Desnim klikom na folder fajlovi_python otvori meni i izaberi **Import**. Prevuci fajlove zone_dostave.csv, dostave_uzorak.csv, kurirske_sluzbe.json, dostave.csv i dostave.parquet, pa potvrdi uvoz. Fajlovi se pojavljuju u folderu, pored notebook-a.
5. U notebook-u citanje_fajlova pokreni ćeliju:

```
import os
print(os.getcwd())
print(sorted(os.listdir(".")))
```

6. Proveri da ispis sadrži imena svih pet fajlova; na spisku je i sam notebook, kao citanje_fajlova.py. Tekući folder je folder u kome je notebook, pa se fajlovi otvaraju samo imenom, npr. open("zone_dostave.csv", encoding="utf-8").

Očekivani rezultat

U folderu fajlovi_python su notebook citanje_fajlova i pet fajlova. os.getcwd() ispisuje putanju tog foldera, koja počinje sa /Workspace/Users/ i završava se sa /fajlovi_python, a os.listdir(".") imena svih pet fajlova i notebook-a.

Ako zapne

- os.listdir(".") ne prikazuje fajlove: notebook nije u folderu fajlovi_python. To se dešava kada se notebook napravi preko **+ New** u meniju levo, jer se tako pravi u ličnom folderu. Napravi notebook unutar foldera, kao u koraku 3.
- FileNotFoundError pri čitanju: fajl nije u istom folderu kao notebook, ili je ime napisano drugačije (mala i velika slova se razlikuju).
- Uvoz fajla ne prolazi: fajl u workspace-u može da ima najviše 500 MB. Zip arhivu Databricks pri uvozu sam raspakuje.

Dodatni izazov

Otvori fajl kurirske_sluzbe.json direktno iz Workspace-a (klikom na njega). Šta se prikazuje?

Rešenje: fajl se otvara u editoru, pa se JSON čita i bez koda. Izmene u editoru se čuvaju same, zato se fajl sa podacima samo čita i ne menja.

Lab 5: Unity Catalog u Catalog Explorer-u

Uz čas: Čas 24 **Trajanje:** 15 min

> **Postavka:** Pre selidbe na lakehouse šef tima traži spisak onoga što već postoji: koje tabele postoje, šta je u njima, ko ih je menjao i od čega je nastala gold tabela iz GATE 1.

Cilj: polaznik u Catalog Explorer-u pronalazi tabelu po katalogu i šemi, čita njene kolone, primer podataka, detalje i istoriju izmena, i na grafu porekla vidi od kojih tabela je nastala gold tabela.

Koraci

1. U meniju levo klikni **Catalog**. Sa leve strane otvara se spisak kataloga.
2. Otvori katalog **workspace**, pa šemu **default**. Prikazuju se tabele i pogledi koje si pravio od prve nedelje kursa.
3. Klikni na tabelu **dostave**. Na strani tabele, na prvoj kartici, pronađi kolonu package_count i pogledaj njen tip.
4. Otvori karticu **Sample Data** i pogledaj nekoliko redova. Za prikaz primera treba compute, pa prvi put može da potraje nekoliko sekundi.
5. Otvori karticu **Details** i pogledaj svojstva tabele, na primer vrstu tabele.
6. Vrati se u šemu default i otvori tabelu **dostave_radna**. Na kartici **History** svaka izmena tabele je posebna verzija; među njima su pravljenje kopije i naredbe UPDATE i DELETE sa Časa 10.
7. Otvori svoju gold tabelu iz GATE 1 (ime joj počinje sa dostave_gold_). Na kartici **Lineage** klikni **See lineage graph** i pogledaj tabele sa leve strane grafa: to su izvori iz kojih je gold tabela nastala.
8. U spisku kataloga otvori katalog **samples** i pogledaj koje šeme ima. To su primeri podataka koje Databricks daje svim korisnicima.

Očekivani rezultat

Polaznik zna gde se vide kolone, primer podataka, detalji i istorija tabele. Na istoriji tabele dostave_radna vide se UPDATE i DELETE sa Časa 10, a na grafu porekla gold tabele vide se tabele iz kojih je napravljena (npr. dostave i zone_dostave).

Ako zapne

- Tabela se ne vidi u spisku: osveži stranicu i proveri da je otvoren katalog workspace i šema default.
- Kartica Sample Data ne prikazuje redove: compute se još pokreće; sačekaj nekoliko sekundi i otvori karticu ponovo.
- Istorija tabele dostave_radna je kraća nego što se očekuje: Databricks čuva istoriju tabele 30 dana, pa starije izmene više nisu na spisku.

- Graf porekla je prazan: tabela nije napravljena upitom nad drugim tabelama, ili gold tabela iz GATE 1 još ne postoji. Otvori poreklo tabele dostave_radna, koja je napravljena iz tabele dostave.

Dodatni izazov

U katalogu samples pronađi tabelu sa vožnjama taksija u Njujorku i napiši upit koji broji njene redove.

Rešenje: tabela je u šemi nyctaxi, pa je njeno puno ime samples.nyctaxi.trips:

```
SELECT COUNT(*) AS broj_voznji FROM samples.nyctaxi.trips;
```

Lab 6: Genie Code u notebook-u

Uz čas: Čas 24 **Trajanje:** 20 min

> **Postavka:** Kolega je pre odlaska na odmor ostavio upit koji ne radi, a menadžer operacija čeka spisak zona u kojima dostave traju najduže. Polaznik koristi Genie Code, uz pravilo "prvo razumi, pa koristi".

Cilj: polaznik koristi Genie Code za ispravku greške, objašnjenje upita i pisanje novog upita, i svaki predlog proverava pre nego što ga zadrži.

Koraci

1. U ličnom folderu napravi nov notebook (**+ New** > **Notebook**), nazovi ga genie_code_proba i u padajućem meniju za jezik, u traci sa menijima, odmah posle menija Help, promeni jezik u SQL.

2. U prvu ćeliju upiši kolegin upit i pokreni ga. Upit javlja grešku, jer se uslov nad agregatom ne piše u WHERE.

```
SELECT pickup_location_id, COUNT(*) AS broj_dostava
FROM workspace.default.dostave
WHERE COUNT(*) > 10000
GROUP BY pickup_location_id;
```

3. Ispod greške klikni **Diagnose error**. Genie Code prikazuje predlog ispravke, sa razlikama u odnosu na stari kod. Klikni **Accept** tek kada možeš da objasniš šta je promenjeno: uslov nad COUNT(*) prelazi u HAVING, posle GROUP BY. Pokreni ispravljen upit.

```
SELECT pickup_location_id, COUNT(*) AS broj_dostava
FROM workspace.default.dostave
GROUP BY pickup_location_id
HAVING COUNT(*) > 10000;
```

4. Klikni u ćeliju sa ispravljenim upitom i pritisni Ctrl+I (na Mac-u Cmd+I). U polje koje se pojavi upiši /explain i pritisni Enter. Pročitaj objašnjenje i uporedi ga sa onim što znaš o GROUP BY i HAVING.

5. Otvori panel Genie Code klikom na ikonicu sa iskricom u gornjem desnom uglu strane (ne na onu u uglu ćelije, koja otvara Genie Code samo za tu ćeliju) i upiši zahtev: "Napiši SQL upit nad dostave_ciste i zone_dostave koji vraća tri zone kupca sa najdužim prosečnim trajanjem dostave, sa nazivom zone i gradom." Pogled i tabelu navedi preko znaka @: kada se upiše @, otvara se spisak objekata iz koga se bira.

6. Kopiraj predloženi upit u novu ćeliju i pokreni ga. Pre nego što ga zadržiš, proveri tri stvari: da li se zona kupca spaja preko dropoff_location_id (a ne preko pickup_location_id, koji je zona magacina), da li se računa prosek kolone trajanje_min i da li je redosled opadajući.

7. Ako neka od tri stvari nije ispunjena, napiši Genie Code-u šta treba promeniti ili ispravi upit ručno. Ispravan upit izgleda ovako:

```
SELECT z.zone AS zona, z.region AS grad, ROUND(AVG(c.trajanje_min), 1) AS prosecno_trajanje_min
FROM workspace.default.dostave_ciste c
JOIN workspace.default.zone_dostave z ON c.dropoff_location_id = z.location_id
GROUP BY z.zone, z.region
ORDER BY prosecno_trajanje_min DESC
LIMIT 3;
```

Očekivani rezultat

Notebook genie_code_proba ima ispravljen kolegin upit sa HAVING, koji vraća četiri zone magacina sa više od 10.000 dostava, i proveren upit sa tri zone kupca sa najdužim dostavama. Na vrhu je zona Aerodrom Nikola Tesla.

Ako zapne

- Nema dugmeta Diagnose error: ćelija nije završila greškom u kodu. Proveri da je upit prepisan tačno i pokreni ga ponovo.
- Predlog koristi pogrešnu tabelu ili kolone koje ne postoje: ponovi zahtev i tabelu izaberi preko znaka @, sa spiska objekata koji se tada otvara.
- Predlog je pogrešan: to se dešava i nije greška polaznika; napiši šta ne valja ili ispravi upit ručno.
- Uputstvo sa interneta pominje Databricks Assistant: to je stari naziv za Genie Code.

Dodatni izazov

Zatraži od Genie Code-a da isti upit napiše u Python-u, preko spark.sql, i proveri da daje iste tri zone.

Rešenje: rezultat mora da bude isti, jer je upit isti, a menja se samo mesto odakle se pokreće. Ako se razlikuje, razlika je gotovo uvek u spajanju ili u filteru, pa se dva upita porede red po red.

Lab 7: Notebook-ovi u GitHub repo-u (Git folder)

Uz čas: Čas 24 **Trajanje:** 30 min

> **Postavka:** Šef tima pregleda sav kod preko pull request-ova, a notebook-ovi su do sada bili van repo-a. Od danas se čuvaju u istom repo-u kao SQL upiti i Python skripte, uz iste grane i preglede.

Cilj: polaznik povezuje Databricks sa GitHub-om, klonira repo data-engineer-kurs u Git folder, pravi granu, radi commit i push notebook-a i posle spajanja pull request-a povlači izmene (Pull).

Koraci

1. Klikni na svoju ikonicu korisnika u gornjem desnom uglu (krug sa početnim slovom imena), izaberi **Settings** i otvori karticu **Linked accounts**.

2. Klikni **Add Git credential**, za provajdera izaberi **GitHub**, izaberi **Link Git account** i klikni **Link**. Na strani GitHub-a koja se otvori klikni **Authorize Databricks**.

3. Otvori stranu za instalaciju aplikacije Databricks na GitHub-u, github.com/apps/databricks/installations/new, i izaberi svoj nalog. Izaberi **Only select repositories**, sa spiska izaberi repo data-engineer-kurs i klikni **Install**. Tako Databricks dobija pristup samo tom repo-u.

4. U meniju levo klikni **Workspace** i otvori lični folder. Klikni **Create** > **Git folder**.

5. U polje **Git repository URL** nalepi adresu repo-a sa GitHub-a (zeleno dugme Code na strani repo-a, kartica HTTPS). Za **Git provider** izaberi **GitHub** i klikni **Create Git folder**. U ličnom folderu pojavljuje se folder data-engineer-kurs, sa svim folderima i fajlovima iz repo-a.

6. Klikni **Git** pored imena foldera data-engineer-kurs. U Git dijalogu klikni **Create branch**, upiši ime grane platforma, kao osnovu ostavi main i klikni **Create**. Zatvori dijalog.

7. Otvori folder data-engineer-kurs i napravi u njemu folder databricks (**Create** > **Folder**), a zatim u folderu databricks nov Python notebook (**Create** > **Notebook**) sa imenom 01_platforma. Proveri u padajućem meniju za jezik, u traci sa menijima, da je jezik Python: nov notebook dobija jezik koji je poslednji korišćen, a na Lab-u 6 to je bio SQL. U prvu ćeliju upiši SQL upit, sa oznakom %sql u prvom redu, a u drugu Python kod, pa pokreni obe ćelije:

```
%sql
SELECT COUNT(*) AS broj_dostava FROM workspace.default.dostave;
```

```
zone = spark.table("workspace.default.zone_dostave")
display(zone)
```

8. Otvori Git dijalog iz notebook-a, klikom na dugme sa imenom grane pored naziva notebook-a. U dijalogu se vidi nov notebook kao izmena. Upiši poruku commit-a "dodat notebook 01_platforma" i klikni **Commit & Push**.

9. Na GitHub-u otvori pull request sa grane platforma ka main, kao na Času 2, i spoji ga.

10. Vrati se u Databricks i u Git dijalogu pređi na granu main. Notebook 01_platforma se ne vidi, jer Git folder još ne zna za spojen pull request. Klikni **Pull**: notebook se pojavljuje.

Očekivani rezultat

Na GitHub-u, u grani main, postoji folder databricks sa notebook-om 01_platforma. U Databricks-u je Git folder data-engineer-kurs na grani main i ima isti sadržaj kao repo na GitHub-u.

Ako zapne

- Push javlja grešku o pristupu: aplikacija Databricks nema pristup repo-u. Na strani za instalaciju aplikacije (korak 3) dodaj repo data-engineer-kurs, pa ponovi **Commit & Push**.
- Push je odbijen jer grana na GitHub-u ima novije izmene: prvo klikni **Pull**, pa ponovo **Commit & Push**.
- Posle spajanja pull request-a notebook se ne vidi u grani main: nije urađen Pull.
- Git folder se ne pravi i javlja grešku o pristupu: nalog nije povezan sa GitHub-om (koraci 1 i 2) ili aplikacija Databricks nema pristup repo-u (korak 3).

Dodatni izazov

Na GitHub-u izmeni README.md direktno u browser-u i sačuvaj izmenu u grani main. Kada se izmena vidi u Git folderu?

Rešenje: tek posle Pull-a. Git folder je kopija repo-a i sam se ne osvežava, zato je pravilo da se pre svakog rada prvo uradi Pull.

Lab 8: Volume raw i dnevni fajlovi

Uz čas: Čas 25 **Trajanje:** 30 min

> **Postavka:** Analitika BrzaKorpe seli se na lakehouse, a kurirske službe od danas svakog jutra šalju fajl sa dostavama. Sirovi fajlovi treba da imaju jedno mesto na platformi pre nego što postanu tabele.

Cilj: polaznik pravi volume raw, otprema u njega fajlove iz arhive sa podacima i notebook-om isporučuje dnevne fajlove prve nedelje januara.

Koraci

1. U meniju levo klikni **Catalog**, otvori katalog **workspace** i izaberi šemu **default**. Klikni **Create** > **Volume**, upiši ime raw, ostavi podrazumevanu vrstu (managed) i klikni **Create**.

2. Otvori volume **raw** i klikni **Upload to this volume**. Izaberi fajl **dostave.parquet** iz raspakovane arhive sa podacima, proveri da je odredište /Volumes/workspace/default/raw i klikni **Upload**.

3. U volume-u raw klikni **Create directory**, upiši ime za_isporuku i klikni **Create**.

4. Otvori folder za_isporuku i u njega otpremi sve fajlove iz foldera dnevno (31 fajl, od dostave_2024-01-01.csv do dostave_2024-01-31.csv), isto kao u koraku 2. Ako dijalog ne primi sve fajlove odjednom, otpremi ih u nekoliko navrata.

5. Otvori Git dijalog foldera data-engineer-kurs, na grani main klikni **Pull**, pa napravi granu isporuka (**Create branch**), kao na Lab-u 7. U folderu databricks napravi Python notebook isporuka (**Create** > **Notebook**) i u prvu ćeliju upiši kod:

```
import os
import shutil

ZA_ISPORUKU = "/Volumes/workspace/default/raw/za_isporuku"
DOSTAVE = "/Volumes/workspace/default/raw/dostave"



def isporuci(datum):
    """Kurirska služba ostavlja fajl za dati dan u folder raw/dostave."""
    ime = f"dostave_{datum}.csv"
    os.makedirs(DOSTAVE, exist_ok=True)
    shutil.copy(os.path.join(ZA_ISPORUKU, ime), os.path.join(DOSTAVE, ime))
    print("isporučen", ime)



for dan in range(1, 8):
    isporuci(f"2024-01-{dan:02d}")
```

6. Pokreni ćeliju. Ispod nje se ispisuje sedam redova, po jedan za svaki isporučen fajl.

7. Dodaj novu ćeliju, upiši proveru i pokreni je. Oznaka %sql u prvom redu menja jezik ćelije.

```
%sql
LIST '/Volumes/workspace/default/raw/dostave/'
```

8. Sačuvaj notebook u repo: u Git dijalogu upiši poruku commit-a "notebook isporuka" i klikni **Commit & Push**. Na GitHub-u otvori i spoji pull request sa grane isporuka ka main, pa u Git dijalogu pređi na main i klikni **Pull**.

Očekivani rezultat

U volume-u raw su fajl dostave.parquet i dva foldera: za_isporuku sa 31 fajlom i dostave sa sedam fajlova, od 1. do 7. januara. Notebook isporuka je u Git folderu i na GitHub-u.

Ako zapne

- Fajl je otišao na pogrešno mesto (drugi volume ili folder): u dijalogu za otpremanje proveri odredište, a pogrešno otpremljen fajl obriši i otpremi ponovo.
- FileNotFoundError u notebook-u: fajl nije u folderu za_isporuku ili mu je ime drugačije. Sadržaj foldera se proverava naredbom LIST, kao u koraku 7.
- Otpremljena je zip arhiva: volume je čuva kao jedan fajl i ne raspakuje je. Otpremaju se raspakovani fajlovi.

Dodatni izazov

Pozovi isporuci("2024-01-07") još jednom, u novoj ćeliji. Koliko je sada fajlova u folderu dostave?

Rešenje: i dalje sedam. shutil.copy prepisuje fajl istog imena, pa ponovljena isporuka istog dana ne pravi nov fajl. Isporuka je idempotentna, kao i upis u mini ETL-u.

Lab 9: Dnevni fajlovi kao tabela, u SQL-u i u Python-u

Uz čas: Čas 25 **Trajanje:** 30 min

> **Postavka:** Menadžer operacija pita koliko dostava stiže u svakom dnevnom fajlu i koja kurirska služba šalje više. Odgovor treba dati odmah, direktno iz fajlova, pre nego što od njih nastanu tabele.

Cilj: polaznik čita sve dnevne fajlove iz volume-a kao jednu tabelu, sa imenom fajla za svaki red, pravi privremeni pogled i isti upit izvršava u SQL-u i u Python-u.

Koraci

1. U Git folderu, u folderu databricks, napravi notebook fajlovi (**Create** > **Notebook**) i u padajućem meniju za jezik, u traci sa menijima, odmah posle menija Help, promeni jezik u SQL.

2. U prvu ćeliju upiši upit i pokreni ga. Posle isporuke prve nedelje rezultat je 18.295.

```
SELECT COUNT(*) AS redova
FROM read_files('/Volumes/workspace/default/raw/dostave/', format => 'csv', header => true);
```

3. Pogledaj nekoliko redova upitom SELECT * FROM read_files(...) LIMIT 5, sa istom putanjom i opcijama. Poslednja kolona je _rescued_data; u ovim fajlovima je prazna.

4. U novu ćeliju upiši privremeni pogled koji svakom redu dodaje ime fajla, pa ga pokreni:

```
CREATE OR REPLACE TEMP VIEW dnevne_dostave AS
SELECT *, _metadata.file_name AS izvorni_fajl
FROM read_files('/Volumes/workspace/default/raw/dostave/', format => 'csv', header => true);
```

5. Izračunaj broj dostava po fajlu i kurirskoj službi:

```
SELECT izvorni_fajl, courier_id, COUNT(*) AS broj
FROM dnevne_dostave
GROUP BY izvorni_fajl, courier_id
ORDER BY izvorni_fajl, courier_id;
```

6. Dodaj ćeliju sa oznakom %python u prvom redu i isti upit uradi iz Python-a: fajlove pročitaj sa spark.read, dodaj ime fajla, napravi privremeni pogled i pokreni isti SQL preko spark.sql.

```
%python
dnevne = (spark.read.format("csv")
          .option("header", True)
          .option("inferSchema", True)
          .load("/Volumes/workspace/default/raw/dostave/")
          .select("*", "_metadata.file_name"))
dnevne.createOrReplaceTempView("dnevne_dostave_py")
display(spark.sql("""
    SELECT file_name AS izvorni_fajl, courier_id, COUNT(*) AS broj
    FROM dnevne_dostave_py
    GROUP BY file_name, courier_id
    ORDER BY izvorni_fajl, courier_id
"""))
```

7. Uporedi rezultate koraka 5 i 6: brojevi moraju da budu isti.

Očekivani rezultat

Notebook fajlovi ima SQL i Python ćelije koje za svaki od sedam fajlova prve nedelje daju isti broj dostava po kurirskoj službi.

Ako zapne

- TABLE_OR_VIEW_NOT_FOUND za dnevne_dostave: ćelija koja pravi privremeni pogled nije pokrenuta u ovoj sesiji; pokreni je ponovo.
- U Python-u se kolone zovu _c0, _c1: nedostaje opcija header.
- U Python-u su sve kolone tekst: nedostaje opcija inferSchema.

Dodatni izazov

U kojim fajlovima ima duplikata? Za svaki fajl uporedi broj redova i broj različitih dostava.

Rešenje:

```
SELECT izvorni_fajl, COUNT(*) AS redova, COUNT(DISTINCT delivery_id) AS dostava
FROM dnevne_dostave
GROUP BY izvorni_fajl
HAVING COUNT(*) > COUNT(DISTINCT delivery_id)
ORDER BY izvorni_fajl;
```

Duplikati su unutar istog fajla: kurirska služba ponekad pošalje isti red dvaput. Bronze sloj ih čuva, a silver ih uklanja.

Lab 10: Alert za loš fajl kurirske službe

Uz čas: Čas 28 **Trajanje:** 15 min

> **Postavka:** Jednog jutra kurirska služba pošalje pokvaren izvoz, pun negativnih cena. Silver loše redove tiho izbaci, pa izveštaji pokažu manje dostava nego što ih je bilo. Za to treba saznati odmah, a ne kada direktorka primeti čudan broj na dashboard-u.

Cilj: polaznik pravi SQL alert koji svakog jutra proverava poslednji dnevni fajl u bronze sloju i šalje obaveštenje kada je procenat loših redova prevelik.

Koraci

1. U meniju levo, u grupi SQL, klikni **Alerts**, pa **Create alert**. Alert ima sopstveni upit: sačuvani upit iz Queries ne može da se iskoristi. Otvara se editor alerta sa privremenim imenom (New Alert i datum); klikni na ime na vrhu i promeni ga u los_fajl.

2. Upiši upit u editor i klikni **Run all**. Ako editor traži SQL warehouse, izaberi **Serverless Starter Warehouse**. Rezultat je jedan broj, procenat loših redova u poslednjem fajlu:

```
SELECT ROUND(100 * AVG(CASE WHEN delivery_fee < 0
                            OR delivery_distance <= 0 OR delivery_distance > 100
                            OR dropoff_datetime <= pickup_datetime
                            OR package_count = 0
                           THEN 1 ELSE 0 END), 1) AS procenat_losih
FROM workspace.default.dostave_bronze
WHERE izvorni_fajl = (SELECT MAX(izvorni_fajl) FROM workspace.default.dostave_bronze);
```

3. U panelu desno, u delu **Condition**, izaberi kolonu procenat_losih, operator > i prag 10 (**Static Value**). Klikni **Test condition**: alert se ne bi aktivirao, jer u običnom fajlu loših redova ima manje od 4%.

4. U delu **Notifications**, u polju **Notify**, potraži svoje korisničko ime i izaberi ga, da obaveštenje stigne na tvoj mejl.

5. U traci iznad editora klikni ikonicu **Schedule**. U prozoru **Edit schedule** ostavi **Day**, podesi vreme na 07:00, ujutru, posle isporuke fajla, i klikni **Save**. Alert se čuva sam, bez posebnog dugmeta; klikni **View alert**.

6. Proba alerta: klikni **Edit**, privremeno spusti prag na 2, pa klikni **View alert** i **Run now**. U delu **Latest status** piše Triggered, u tabeli **History** vidi se na koji mejl je poslato obaveštenje, a poruka stiže u sanduče.

7. Na isti način vrati prag na 10 i ponovo klikni **Run now**. Stanje se vraća na OK.

Očekivani rezultat

Alert los_fajl je na listi **Alerts** u stanju OK, sa rasporedom svakog dana u 07:00. U tabeli **History** vidi se proba sa pragom 2, u stanju Triggered, i mejl na koji je obaveštenje poslato.

Ako zapne

- Stanje Error: upit ne radi, najčešće zato što tabela dostave_bronze još ne postoji ili je ime pogrešno. Pokreni upit u SQL Editor-u i ispravi ga.

- Obaveštenje ne stiže: proveri da si u koraku 4 dodao sebe i pogledaj folder za neželjenu poštu. U tabeli **History**, u koloni **Failed destinations**, vidi se da li slanje nije uspelo.
- Uputstvo sa interneta traži sačuvani upit iz Queries: to je stara verzija alerta, koja je na strani Alerts na kartici **Legacy alerts**. Lab koristi novu verziju, u kojoj je upit deo alerta.
- Alert troši kvotu Free Edition-a pri svakom pokretanju: zato se pokreće jednom dnevno, a ne na nekoliko minuta.

Dodatni izazov

Napravi drugi alert koji javlja kada poslednji isporučeni fajl ima manje od 1.000 redova, jer to znači da je izvoz prekinut na pola.

Rešenje: upit vraća broj redova poslednjeg fajla, a uslov je da je vrednost manja od 1000:

```
-- uslov alerta: redova < 1000
SELECT COUNT(*) AS redova
FROM workspace.default.dostave_bronze
WHERE izvorni_fajl = (SELECT MAX(izvorni_fajl) FROM workspace.default.dostave_bronze);
```

Prag se bira prema podacima: fajl najmirnijeg dana u januaru, 1. januara, ima 1.167 redova, pa bi prag od 1.000 za taj dan bio blizu granice, a za običan dan daleko ispod.

Lab 11: Dashboard za menadžment

Uz čas: Čas 29 **Trajanje:** 25 min

> **Postavka:** Direktorka traži jednu stranicu sa brojevima o dostavama, koju može da otvori na telefonu pre sastanka, bez SQL-a. Gold tabele iz prošlog časa imaju sve što joj treba.

Cilj: polaznik pravi dashboard nad gold tabelama, sa linijom broja dostava po danu, stubićima trajanja po vremenu i filterom po gradu, i objavljuje ga.

Koraci

1. Klikni **+ New** > **Dashboard**. Otvara se nov, prazan dashboard. Klikni na njegov naziv, veliki naslov gore levo koji počinje sa New Dashboard, i upiši Dostave, januar 2024. Ispod naslova je i kartica stranice dashboard-a, koja ima svoje ime; nju ne treba menjati.

2. Otvori karticu **Data** i klikni **Add data**. U prozoru se nude samo nedavno korišćene tabele, pa u polje **Search** upiši dostave_gold i označi **dostave_gold_dnevno** i **dostave_gold_vreme** (ako je neka druga tabela već označena, ukloni je sa spiska izabranih), pa klikni **Add**. Svaka tabela postaje jedan skup podataka.

3. Vrati se na stranu dashboard-a, na donjoj traci klikni ikonicu grafikona (**Add a visualization**) i postavi okvir na platno. U podešavanjima izaberi skup podataka dostave_gold_dnevno, vrstu **Line**, na X osu kolonu dan, na Y osu zbir kolone broj_dostava, a za boju kolonu grad (pored **Color** klikni +). Na X osi Databricks dane sam grupiše po mesecu, pa piše MONTHLY(dan); promeni grupisanje na dnevno, da piše DAILY(dan).

4. Dodaj drugu vizuelizaciju: skup podataka dostave_gold_vreme, vrsta **Bar**, na X osi kolona vreme, na Y osi kolona prosecno_trajanje_min.

5. Dodaj treću vizuelizaciju: skup podataka dostave_gold_dnevno, vrsta **Counter**, sa zbirom kolone broj_dostava. To je ukupan broj dostava u prikazanom periodu, prikazan skraćeno (npr. 96.7K).

6. Dodaj filter: na donjoj traci klikni ikonicu filtera (**Add a filter (field/parameter)**), postavi ga iznad vizuelizacija i za polje izaberi kolonu grad iz skupa podataka dostave_gold_dnevno.

7. Izaberi Novi Sad u filteru i proveri da se linija i brojač promene.

8. Gore desno klikni **Publish** (plavo dugme pored njega služi za deljenje, a ne za objavljivanje). U prozoru ostavi podrazumevanu opciju **Share data permission (default)** i klikni **Publish**. Objavljena verzija se otvara dugmetom **View published**.

Očekivani rezultat

Objavljen dashboard sa tri vizuelizacije i filterom po gradu. Linija pokazuje pad broja dostava 1., 2. i 7. januara, na praznike, a stubići pokazuju da dostave po snegu traju najduže.

Ako zapne

- U podešavanjima vizuelizacije nema kolone: izabran je pogrešan skup podataka; skup se bira pre kolona.
- Linija ima jednu tačku: na X osi piše MONTHLY(dan), pa su svi dani sabrani u jedan mesec; promeni grupisanje na dnevno. Ista slika se dobija i kada je na X osi kolona grad umesto kolone dan.
- Brojač i linija ne reaguju na filter: filter je vezan za drugi skup podataka. Filter po gradu deluje samo na vizuelizacije iz skupa koji ima tu kolonu.

Dodatni izazov

Dodaj vizuelizaciju sa procentom kasnih dostava po danu za svaki grad. Koji grad ima najviše dana sa procentom iznad 10?

Rešenje: skup podataka dostave_gold_dnevno, vrsta Line, X osa dan, Y osa prosek kolone procenat_kasnih, boja grad. Odgovor se čita sa grafikona, a proverava upitom nad istom tabelom:

```
SELECT grad, COUNT(*) AS dana_iznad_10
FROM workspace.default.dostave_gold_dnevno
WHERE procenat_kasnih > 10
GROUP BY grad
ORDER BY dana_iznad_10 DESC;
```

Najviše takvih dana ima Niš: sa fajlovima do 21. januara procenat kasnih je iznad 10 u 16 od 21 dana, a za ceo januar u 24 od 31.

Lab 12: Genie Agent, pitanja o dostavama na običnom jeziku

Uz čas: Čas 29 **Trajanje:** 20 min

> **Postavka:** Analitičarka iz marketinga ne piše SQL, a svaki dan ima nova pitanja o dostavama. Umesto da čeka data tim, pitanja postavlja Genie Agent-u, a data tim se stara da su odgovori tačni.

Cilj: polaznik pravi Genie Agent nad gold tabelama, daje mu uputstva o značenju podataka i proverava njegove odgovore preko generisanog SQL-a.

Koraci

1. U meniju levo, u grupi SQL, klikni **Genie Agents**, pa **New**. Izaberi tabele **dostave_gold_dnevno** i **dostave_gold_vreme** iz kataloga workspace, šema default, i klikni **Create**.

2. Klikni **Configure**. Na kartici **About** Genie Agent sam predlaže opis podataka (**Genie Suggested Description**). Pročitaj ga pre nego što ga prihvatiš, jer predlog lako pročita kolonu vreme kao doba dana, a u tabeli su to vremenske prilike. Tačan opis prihvati sa **Accept**, a pogrešan prvo ispravi preko **Edit**.

3. Na kartici **Instructions**, u polje **General Instructions**, upiši: "Iznosi su u dinarima. Grad dostave je grad magacina. Dostava kasni kada traje duže od obećanog vremena kurirske službe. Vreme je kategorija: suvo, kiša ili sneg." Klikni **Save**.

4. Postavi pitanje: "Koliko je dostava bilo u svakom gradu?" Klikni **Show code** iznad grafikona i proveri da upit sabira kolonu broj_dostava (SUM), a ne broji redove tabele.

5. Postavi pitanje: "Da li sneg produžava dostave?" Uporedi odgovor sa tabelom dostave_gold_vreme. Za jedan odgovor Genie Agent ponekad napravi više upita, i svaki ima svoj **Show code**. Pogledaj svaki: upit u koji je ručno upisan broj iz prethodnog rezultata (npr. 26,2 min po suvom vremenu, upisano direktno u formulu) daje tačan odgovor danas, a pogrešan kada stignu novi podaci.

6. Na dnu svakog tačnog odgovora, uz pitanje **Is this useful?**, klikni **Yes**. Ako odgovor nije tačan, klikni **No**, dopuni uputstva i postavi pitanje ponovo, preciznije. Dugme **Request Review** je za analitičarku kada nije sigurna u odgovor: zahtev vidi data tim, kao autor Genie Agent-a, na kartici **Monitor**.

Očekivani rezultat

Genie Agent sa dve gold tabele i uputstvima, i dva odgovora čiji je SQL proveren. Zbir dostava po gradovima jednak je broju redova silver sloja (96.700), a odgovor o snegu pokazuje da dostave po snegu traju duže nego po suvom (39,4 naspram 26,2 min). Tačni odgovori su označeni sa **Yes**.

Ako zapne

- Odgovor broji redove umesto da sabira broj_dostava: u tabeli dostave_gold_dnevno jedan red je dan u gradu, a ne jedna dostava. Dodaj to u uputstva i ponovi pitanje.

- Genie Agent kaže da nema podataka: tabela nije izabrana u koraku 1. Dodaj je u **Configure**, na kartici **Sources**.
- Odgovor se razlikuje od upita u SQL Editor-u: pitanje je dvosmisleno (npr. "najbolji grad"); pitanje se preformuliše, a dobar upit se čuva kao primer.

Dodatni izazov

Sačuvaj proveren upit za procenat kasnih dostava po gradu kao primer (kartica **Examples**), pa pitanje postavi drugim rečima. Da li Genie Agent sada koristi tvoj upit?

Rešenje: primer upita je SQL koji je data tim proverio; Genie Agent ga koristi kada prepozna isto pitanje, i kada je postavljeno drugim rečima. Upit za primer:

```
SELECT grad,
       ROUND(SUM(procenat_kasnih * broj_dostava) / SUM(broj_dostava), 1) AS procenat_kasnih
FROM workspace.default.dostave_gold_dnevno
GROUP BY grad
ORDER BY procenat_kasnih DESC;
```

Procenat za ceo mesec se računa kao prosek dnevnih procenata ponderisan brojem dostava, jer dan sa više dostava treba da ima veću težinu. Običan prosek dnevnih procenata daje pogrešan broj.

Lab 13: Job koji pokreće tok sam

Uz čas: Čas 29 **Trajanje:** 35 min

> **Postavka:** Fajl kurirske službe stiže svakog jutra, a niko ne želi da svakog jutra ručno pokreće četiri notebook-a. Tok treba da radi sam, redom, i da stane čim nešto pukne.

Cilj: polaznik pravi job sa četiri zadatka (isporuka, bronze, silver, gold) i zavisnostima među njima, prosleđuje datum kao parametar, pokreće job i proverava rezultat, a zatim dodaje raspored i pauzira ga.

Koraci

1. U Git folderu otvori notebook isporuka i sadržaj prve ćelije zameni kodom sa časa: widget-i od i do, pa isporuka svih dana od prvog do drugog datuma. Pokreni notebook ručno jednom, sa podrazumevanim vrednostima, i proveri da je isporučen fajl za 22. januar.

```
import os
import shutil
from datetime import date, timedelta

dbutils.widgets.text("od", "2024-01-22")
dbutils.widgets.text("do", "2024-01-22")

ZA_ISPORUKU = "/Volumes/workspace/default/raw/za_isporuku"
DOSTAVE = "/Volumes/workspace/default/raw/dostave"



def isporuci(datum):
    """Kurirska služba ostavlja fajl za dati dan u folder raw/dostave."""
    ime = f"dostave_{datum}.csv"
    os.makedirs(DOSTAVE, exist_ok=True)
    shutil.copy(os.path.join(ZA_ISPORUKU, ime), os.path.join(DOSTAVE, ime))
    print("isporučen", ime)



dan = date.fromisoformat(dbutils.widgets.get("od"))
kraj = date.fromisoformat(dbutils.widgets.get("do"))
while dan <= kraj:
    isporuci(dan.isoformat())
    dan += timedelta(days=1)
```

2. U meniju levo klikni **Jobs & Pipelines**, pa **Create** > **Job**. Klikni na naziv job-a na vrhu strane i upiši dostave_dnevno.

3. Izaberi vrstu zadatka **Notebook**. U polje **Task name** upiši isporuka, za **Source** ostavi **Workspace**, u polju **Path** izaberi notebook isporuka iz Git folder-a i klikni **Confirm**, pa **Save task**. Compute je serverless i ne podešava se.

4. Klikni **Add task** > **Notebook** i dodaj zadatak bronze sa notebook-om 01_bronze. U polju **Depends on** mora da stoji isporuka. Klikni **Save task**.

5. Na isti način dodaj zadatke silver (notebook 02_silver, zavisi od bronze) i gold (notebook 03_gold, zavisi od silver).

6. U panelu **Job details** klikni **Edit parameters** i dodaj dva parametra: od sa vrednošću 2024-01-22 i do sa istom vrednošću. Klikni **Save**.

7. Klikni **Run now**. Otvori karticu **Runs** i klikni na vreme početka pokretanja: vidi se graf sa četiri zadatka, koji jedan za drugim postaju zeleni.

8. Pokretanje traje nekoliko minuta. Kada završi, u SQL Editor-u proveri da u bronze sloju postoje redovi iz fajla za 22. januar i da su oba sloja porasla: bronze sada ima 68.894 redova, a silver 66.359.

```
SELECT (SELECT COUNT(*) FROM workspace.default.dostave_bronze) AS bronze,
       (SELECT COUNT(*) FROM workspace.default.dostave_bronze
         WHERE izvorni_fajl = 'dostave_2024-01-22.csv') AS iz_fajla_22,
       (SELECT COUNT(*) FROM workspace.default.dostave_silver) AS silver;
```

9. U panelu **Job details** klikni **Add trigger**, za **Trigger type** izaberi **Scheduled**, podesi pokretanje jednom dnevno i klikni **Save**. Odmah zatim, u delu **Schedules & Triggers**, klikni **Pause**: raspored ostaje podešen, ali se job ne pokreće sam dok se raspored ponovo ne uključi.

Očekivani rezultat

Job dostave_dnevno ima četiri zadatka povezana zavisnostima i dva parametra. Jedno pokretanje je uspešno, u bronze sloju postoji fajl za 22. januar, a raspored je podešen i pauziran.

Ako zapne

- Ručno pokretanje u koraku 1 javlja grešku da widget ne postoji: u notebook-u nedostaje dbutils.widgets.text za od ili do. U job-u parametar stiže i bez toga, pa kada zadatak isporuka javi istu grešku, ime parametra job-a se razlikuje od imena widget-a (od i do, malim slovima).
- Zadaci bronze, silver i gold se pokreću istovremeno: nije podešeno Depends on. Svaki zadatak zavisi od prethodnog.
- Pokretanje dugo stoji na početku: serverless compute se pokreće; prvo pokretanje dana može da traje i nekoliko minuta.
- Zadatak je crven: klikni na njega u grafu i pročitaj grešku ispod ćelije koja je pukla. Posle ispravke notebook-a job se pokreće ponovo, isti tok je idempotentan.

Dodatni izazov

Pokreni job ponovo za isti dan, 22. januar. Da li se broj redova u slojevima promenio?

Rešenje: nije. Isporuka prepisuje isti fajl, a bronze, silver i gold se prave iznova iz svih fajlova u folderu, pa ponovljeno pokretanje daje isti rezultat. Zato se posle greške job može pokrenuti ponovo bez straha od duplikata.

Lab 14: Lakeflow pipeline sa proverama kvaliteta

Uz čas: Čas 34 **Trajanje:** 35 min

Postavka: Silver iz SQL-a tiho izbacuje loše redove, a koliko je izbacilo koje pravilo zna samo onaj ko napiše poseban upit. Šef data tima hoće da se to vidi posle svakog pokretanja, bez dodatnog koda, i da tok sam vodi računa o tome koji su fajlovi već učitani.

Cilj: polaznik pravi Lakeflow pipeline sa streaming tabelom za bronze i materijalizovanim pogledom za silver, zadaje pravila čišćenja kao provere kvaliteta (expectations), pokreće pipeline i čita koliko je redova izbacilo svako pravilo.

Koraci

1. U SQL Editor-u napravi šemu za tabele pipeline-a. Pipeline sam pravi i održava svoje tabele, pa se one drže odvojeno od tabela koje puni job, u posebnoj šemi, sa istim imenima slojeva:

```
CREATE SCHEMA IF NOT EXISTS workspace.lakeflow;
```

2. Klikni **+ New** > **ETL pipeline**. Otvara se editor pipeline-a, sa imenom koje počinje sa New Pipeline i datumom. Klikni na ime i upiši dostave_lakeflow.

3. Desno od imena izaberi podrazumevani katalog workspace i šemu lakeflow. Sve tabele iz pipeline-a nastaju u toj šemi.

4. Editor je otvorio fajl my_transformation. U padajućem meniju za jezik izaberi **SQL** i upiši bronze sloj kao streaming tabelu. Ona čita fajlove Auto Loader-om, pa svaki dnevni fajl učitava tačno jednom:

```
CREATE OR REFRESH STREAMING TABLE dostave_bronze
AS SELECT *,
          _metadata.file_name AS izvorni_fajl,
          current_timestamp() AS vreme_ucitavanja
FROM STREAM read_files('/Volumes/workspace/default/raw/dostave/', format => 'csv', header => true);
```

5. Ispod, u istom fajlu, dodaj silver kao materijalizovani pogled. Upit je isti kao u notebook-u 02_silver, a pravila čišćenja iz WHERE su sada provere kvaliteta: svaka ima ime, a ON VIOLATION DROP ROW znači da se red koji je ne ispunjava izbacuje i broji.

```
CREATE OR REFRESH MATERIALIZED VIEW dostave_silver (
  CONSTRAINT paketi_pozitivni EXPECT (package_count > 0) ON VIOLATION DROP ROW,
  CONSTRAINT cena_nije_negativna EXPECT (delivery_fee >= 0) ON VIOLATION DROP ROW,
  CONSTRAINT distanca_ispravna EXPECT (delivery_distance > 0 AND delivery_distance <= 100) ON
VIOLATION DROP ROW,
  CONSTRAINT predaja_posle_preuzimanja EXPECT (dropoff_datetime > pickup_datetime) ON VIOLATION DROP
ROW
)
AS SELECT DISTINCT delivery_id, courier_id, pickup_datetime, dropoff_datetime,
          COALESCE(package_count, 1) AS package_count, delivery_distance,
          pickup_location_id, dropoff_location_id, payment_type,
          delivery_fee, handling_fee, extra_fee, order_value,
          ROUND(timestampdiff(SECOND, pickup_datetime, dropoff_datetime) / 60, 1) AS trajanje_min
FROM dostave_bronze;
```

6. Klikni **Run pipeline**. Prvo pokretanje traje nekoliko minuta, jer se pokreće serverless compute i učitava ceo mesec. U grafu desno pojavljuju se dostave_bronze i dostave_silver, povezane u redosledu u kome nastaju. Pipeline je sam odredio da silver čeka bronze, iz toga što silver čita dostave_bronze.

7. Kada pokretanje završi, u meniju levo klikni **Jobs & Pipelines**, pa ime pipeline-a dostave_lakeflow. Klikni na dostave_silver u grafu i u panelu desno otvori karticu **Data quality**: za svaku proveru vidi se koliko redova je nije ispunilo.

8. U SQL Editor-u proveri broj redova i kontrolni zbir silver sloja iz pipeline-a. Brojevi moraju da budu isti kao za dostave_silver iz SQL-a i iz PySpark-a:

```
SELECT COUNT(*) AS redova,
       SUM(delivery_id) AS zbir_id,
       ROUND(SUM(delivery_fee), 2) AS zbir_cena,
       ROUND(SUM(trajanje_min), 1) AS zbir_trajanje
FROM workspace.lakeflow.dostave_silver;
```

9. Klikni **Run pipeline** još jednom. Streaming tabela ne nalazi nove fajlove, pa bronze ne dobija nijedan red, a silver ostaje isti. Pipeline nema raspored i ne pokreće se sam, pa ne troši kvotu dok ga ponovo ne pokreneš.

Očekivani rezultat

Pipeline dostave_lakeflow ima dve tabele u šemi workspace.lakeflow. Kada su u folderu raw/dostave svi dnevni fajlovi januara, bronze ima 100.400 redova, a silver 96.700, sa istim kontrolnim zbirom kao silver iz SQL-a: 4.834.732.086, 29.401.503,02 i 2.636.665,1. Kartica Data quality pokazuje broj izbačenih redova za sve četiri provere.

Ako zapne

- Bronze ima manje redova nego što se očekuje: u folderu raw/dostave nisu svi dani januara. Isporuči ih job-om sa Časa 29 i ponovo pokreni pipeline; streaming tabela učitava samo fajlove koji su stigli u međuvremenu.
- Pipeline javlja grešku za tabelu dostave_bronze već pri prvom pokretanju: najverovatnije nije izabrana šema lakeflow, pa pipeline pokušava da napravi tabelu u šemi default, gde tabelu sa istim imenom puni job. Vrati se na korak 3.
- Pokretanje ne kreće jer je aktivan drugi pipeline: Free Edition dozvoljava jedan aktivan pipeline iste vrste. Sačekaj da prethodno pokretanje završi.
- Silver javlja da kolona package_count ne postoji: bronze je učitan bez header => true, pa se kolone zovu _c0, _c1 i tako dalje. Ispravi upit za bronze i klikni **Run pipeline with full table refresh**, da se obe tabele naprave iznova.

Dodatni izazov

Promeni proveru cena_nije_negativna tako da umesto DROP ROW ima ON VIOLATION FAIL UPDATE, i pokreni pipeline. Šta se desilo sa tabelom dostave_silver?

Rešenje: pokretanje pada na proveri za cenu, jer u podacima ima 800 redova sa negativnom cenom. Izmena tabele se poništava, pa dostave_silver ostaje ista kao posle prethodnog pokretanja. FAIL UPDATE se koristi za pravilo koje nikad ne sme da bude prekršeno, na primer

da delivery_id nije prazan, a za loše redove koji se u izvozu kurirskih službi očekuju ide DROP ROW. Vrati DROP ROW i pokreni pipeline ponovo.

Lab 15: Povezivanje dbt-a sa Databricks-om

Uz čas: Čas 36 **Trajanje:** 20 min

> **Postavka:** dbt radi na računaru polaznika, a SQL izvršava SQL warehouse na Databricks-u. Da bi se povezao, dbt-u trebaju adresa workspace-a, putanja do warehouse-a i token. Token je lozinka: ko ga ima, radi u workspace-u u ime vlasnika tokena.

Cilj: polaznik pravi lični token za dbt, pronalazi podatke za povezivanje sa SQL warehouse-om, upisuje ih u fajl .env koji ne ide u repo i proverava vezu komandom dbt debug.

Koraci

1. U meniju levo, u grupi SQL, klikni **SQL Warehouses** i otvori warehouse iz spiska (Free Edition ima jedan). Na kartici **Connection details** prepiši vrednosti **Server hostname** i **HTTP path**.

2. Klikni na svoju ikonicu korisnika u gornjem desnom uglu (krug sa početnim slovom imena) i izaberi **Settings**, pa **Developer**. Pored **Access tokens** klikni **Manage**, pa **Generate new token**.

3. Za ime tokena upiši dbt, a za trajanje 90 dana, da token traje do kraja kursa. Za vrstu tokena izaberi **BI Tools**, jer dbt radi preko SQL warehouse-a, i klikni **Generate**.

4. Token se prikazuje samo jednom. Kopiraj ga odmah u fajl .env u folderu dostave_dbt, zajedno sa vrednostima iz koraka 1, pa klikni **Done**. Dok je token na ekranu, na Zoom-u se ne deli ceo ekran.

```
DATABRICKS_HOST=<Server hostname>
DATABRICKS_HTTP_PATH=<HTTP path>
DATABRICKS_TOKEN=<token iz koraka 4>
```

5. U terminalu, u folderu dostave_dbt, sa aktiviranim virtuelnim okruženjem, pokreni dbt debug. Prvo povezivanje može da potraje, jer se warehouse pokreće.

6. Otvori fajl .gitignore u folderu dostave_dbt i proveri da je u njemu red .env: dbt init ga je dodao, pa se .env neće poslati u repo kada projekat bude u Git-u.

Očekivani rezultat

dbt debug na kraju ispisuje All checks passed!, a .env je naveden u .gitignore. Token postoji samo u tom fajlu i na strani Access tokens, gde se vidi njegovo ime i datum isteka.

Ako zapne

- dbt debug javlja Env var required but not provided: .env nije u folderu iz koga se pokreće dbt, ili se ime promenljive u njemu razlikuje od imena u profiles.yml.
- Greška pri povezivanju ili odbijen pristup: token je prepisan sa razmakom ili bez nekog znaka, ili je u host upisana cela adresa, sa protokolom na početku. Host se upisuje tačno kao u polju Server hostname.
- Povezivanje traje duže od jednog minuta: warehouse se pokreće posle pauze. Sačekaj i pokreni dbt debug ponovo.

- dbt javlja da token nema potrebnu dozvolu (scope): napravi nov token vrste **Other APIs**, izaberi opseg sql i zameni stari token u .env.
- dbt --version javlja da je dbt-core zastareo: to je očekivano, dbt-databricks sam bira verziju dbt-core-a. dbt-core se ne nadograđuje ručno.

Dodatni izazov

Kolega je greškom poslao token u javni repo, pa ga je u sledećem commit-u obrisao. Da li je time problem rešen?

Rešenje: nije. Stari commit je i dalje u istoriji repo-a, a javan repo mogu da pročitaju i automatski programi koji traže tokene. Token se odmah opoziva na strani Access tokens, pravi se nov i upisuje samo u .env. Posle opoziva stari token više ništa ne otvara, pa commit u istoriji više nije opasan.

Lab 16: AI Playground sa alatom

Uz čas: Čas 45 **Trajanje:** 25 min

> **Postavka:** Menadžer operacija bi da pita koliko je dostava bilo juče u Beogradu, a da ne piše SQL. Pre nego što se napiše ijedna linija koda, u Playground-u se proba šta model ume sam, a šta tek uz alat.

Cilj: polaznik u AI Playground-u bira model, piše sistemski prompt, poredi dva modela na istom pitanju i dodaje Unity Catalog funkciju kao alat, pa vidi razliku između odgovora iz glave i odgovora iz podataka.

Koraci

1. U meniju levo, u grupi AI/ML, klikni **Playground**.
2. U padajućem meniju za model, na vrhu strane, izaberi model sa oznakom **Tools enabled**. Samo takav model ume da poziva alate.
3. U polje za sistemski prompt upiši ulogu AI asistenta: pomaže data timu BrzaKorpe, odgovara kratko, na srpskom jeziku, latinicom, i nikad ne izmišlja brojeve o dostavama.
4. Postavi pitanje: Koliko je dostava bilo u Beogradu 5. januara 2024? Zabeleži odgovor.
5. Gore desno klikni znak plus, da dodaš drugi model za poređenje, u oba prozora označi **Sync**, pa postavi isto pitanje: oba modela ga dobijaju u isto vreme.
6. Kada na času nastane funkcija dostave_za_dan, zatvori poređenje i u prvom modelu klikni **Tools**, pa **+ Add tool**. Izaberi **UC Function** i funkciju workspace.default.dostave_za_dan, pa ponovi pitanje iz koraka 4.

Očekivani rezultat

Bez alata model kaže da podatka nema ili izmisli broj. Sa alatom u razgovoru se vidi poziv funkcije dostave_za_dan, sa danom i gradom kao argumentima, i njen rezultat, a broj dostava u odgovoru je isti kao u tabeli dostave_gold_dnevno za 5. januar i Beograd.

Ako zapne

- Nijedan izabrani model ne nudi alate: model nema oznaku Tools enabled; izaberi drugi iz spiska.
- Odgovor stiže sa greškom 429: pređeno je ograničenje broja zahteva na Free Edition-u. Sačekaj nekoliko minuta i pošalji pitanje ponovo; ako se greška ponavlja, nastavi kasnije.
- Funkcija se ne vidi među alatima: pravi se na času, u SQL Editor-u ili notebook-u; posle pravljenja osveži stranu Playground-a.
- Model poziva alat sa datumom u drugom obliku (npr. 5.1.2024): u COMMENT-u argumenta napiši oblik datuma, kao u primeru sa časa.

Dodatni izazov

Dodaj i funkciju kasnjenje_sluzbe kao drugi alat i pitaj koja kurirska služba češće kasni. Koliko puta model poziva alat?

Rešenje: model poziva kasnjenje_sluzbe dva puta, jednom za Strelu i jednom za Kompas, pa poredi rezultate: Kompas kasni u 14,1% dostava, a Strela u 8,0%. Iz opisa oba alata model sam zaključuje koji da pozove, pa jasni opisi funkcija i argumenata vrede više od dugačkog sistemskog prompta.

Prilog: ograničenja Databricks Free Edition-a

Ovo su ograničenja Free Edition-a i serverless compute-a koja utiču na lab-ove, prema zvaničnoj dokumentaciji iz septembra 2026. Isti spisak, uz linkove na dokumentaciju, nalazi se i u prilogu plana i programa nastave.

Compute: koristi se isključivo serverless okruženje; nije dozvoljeno kreiranje prilagođenih klastera; komunikacija sa spoljnim internet servisima ograničena je na pouzdane (odobrene) domene.

SQL warehouse: jedan, veličine 2X-Small.

Jobs: najviše 5 istovremenih task-ova po nalogu.

Lakeflow pipeline: jedan aktivan po tipu.

Model serving: bez GPU, bez provisioned throughput, uz ograničen broj aktivnih endpoint-a.

AI Search: jedan endpoint, jedna search jedinica, bez Direct Vector Access.

Apps: do 3 po nalogu, a aplikacija radi najviše 24 sata posle pokretanja.

Nema: R i Scala, Knowledge Assistant, online tabele, clean rooms i sopstvene lokacije za skladištenje (custom storage).

Nalog: jedan workspace i metastore; samo za nekomercijalnu upotrebu; prijava jednokratnim kodom poslatim mejlom (OTP), preko Google ili Microsoft naloga, bez jedinstvene prijave preko firme (SSO) i bez automatskog upravljanja korisnicima (SCIM).

Kvota: potrošnja se ograničava pravilom poštenog korišćenja (fair use). Kada se dnevna kvota potroši, compute se gasi do kraja dana, a u krajnjem slučaju i do kraja meseca. Zato se posle svakog lab-a sa rasporedom (job, alert) raspored pauzira. Nalog koji se dugo ne koristi može da bude obrisan, pa se sav kod čuva i u Git repo-u, a ne samo u workspace-u.

Ograničenja serverless compute-a

Pored ograničenja naloga, serverless compute ima i svoja ograničenja. Ona koja se vide u primerima kursa su sledeća:

- Podržan je samo DataFrame API (Spark Connect). RDD API nije podržan, pa ne radi ni df.rdd.
- Metode cache(), persist(), unpersist() i checkpoint(), kao i SQL naredbe CACHE TABLE i UNCACHE TABLE, prekidaju izvršavanje greškom.
- Spark UI nije dostupan; umesto njega se koristi query profile.
- ANSI SQL režim je podrazumevan: deljenje nulom i neispravna promena tipa prekidaju upit greškom, umesto da vrate NULL.
- Streaming radi samo sa trigger-om availableNow (i zastarelim once); processingTime i continuous nisu podržani.
- Globalni privremeni pogledi (global temp views) nisu podržani, a pristup DBFS-u je ograničen, pa se fajlovi čuvaju u Unity Catalog volume-ima.
- JAR biblioteke, init skripte i većina Spark podešavanja nisu dostupni.

Prilog: stari i novi nazivi u Databricks-u

Databricks povremeno menja nazive delova platforme. Stariji video-kursevi i tutorijali zato često koriste nazive koji se u meniju više ne vide. U lab-ovima se koriste nazivi iz septembra 2026.

Stari naziv	**Današnji naziv**
Databricks Assistant	Genie Code
Genie space (AI/BI Genie)	Genie Agents
Workflows	Jobs & Pipelines (Lakeflow Jobs)
Delta Live Tables (DLT)	Lakeflow pipelines
Vector Search	AI Search